CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

PROJECT REPORT
ON
Telephone Contact List Using Linked List in C

Subject: Data Structures Using C

SUBJECT CODE:25CAT-151

SUBMITED BY:

Name : Muskan **UID : 25BCD10182**

Branch : BCA(Data Science) **Group: A**

Section:BCD3 **Semester:2nd**

SUBMITED TO:

Name : Kaushlendra Tiwari **D.O.SUBMITION : 10thApril,2026**

Desigination: Assistant professor

Sign:_____

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

INDEX

1. **Candidate Declaration**
2. **Acknowledgement**
3. **Introduction to Data Structures**
4. **Introduction to Arrays and Structures**
5. **Why Linked List?**
6. **Types of Linked List**
7. **Basic Operations on Linked List**
8. **Title of the Project**
9. **Objectives of the Project**
10. **Problem Specification / Need of the Project**
11. **Features of the System**
12. **Algorithm & Flowchart**
13. **Implementation – C Source Code**
14. **Output / Screenshots**
15. **Applications**
16. **Future Scope**
17. **Conclusion**
18. **References**

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Candidate Declaration

I hereby declare that the minor project report entitled **"Telephone Contact List Using Linked List in C"** submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) Data Science at Chandigarh University is a genuine and authentic record of my own work carried out by me during the academic session 2025–2026 under the guidance and supervision of my faculty members.

I further declare that the work presented in this project report is original and has been completed by me with sincere effort and dedication. This project has not been submitted, either in part or in full, to any other university, institution, or organization for the award of any degree, diploma, or certificate. All the information, data, and materials included in this report have been collected, analyzed, and presented by me, and proper references have been given wherever required.

The main objective of this project is to develop a telephone contact management system using the C programming language and linked list data structure. During the development of this project, I have applied various programming concepts such as structures, pointers, dynamic memory allocation, and linked list operations including insertion, deletion, searching, and updating of records. This project has helped me gain practical knowledge and a deeper understanding of how data structures are implemented in real-life applications.

I also confirm that all the coding, testing, and documentation work has been carried out by me. Any external assistance received during the preparation of this project has been properly acknowledged in the respective section. I have followed all academic guidelines and ethical standards while completing this project.

I take full responsibility for the accuracy, authenticity, and originality of the work presented in this report. In case any discrepancy, error, or violation of academic rules is found, I shall be solely responsible for the same.

This project has been prepared purely for academic purposes and to enhance my technical knowledge, problem-solving ability, and programming skills. I believe that this work reflects my learning and effort during the course of my study.

Submitted By: Muskan
UID: 25BCD10182
Course: BCA Data Science

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Acknowledgement

I would like to express my sincere gratitude and heartfelt thanks to all those who have directly or indirectly contributed to the successful completion of my minor project titled **"Telephone Contact List Using Linked List in C"**. This project has been an important part of my academic journey, and it would not have been possible without the support, guidance, and encouragement of many individuals.

First and foremost, I would like to thank **Chandigarh University** for providing me with the valuable opportunity to work on this project as a part of my curriculum. The university has always encouraged practical learning and innovation, which motivated me to apply theoretical concepts in a real-world scenario.

I am highly thankful to the **Department of Computer Science & Engineering** for providing a supportive academic environment, necessary resources, and technical knowledge required for the successful completion of this project. The department has played a key role in enhancing my understanding of programming and data structures.

I would like to express my deep sense of gratitude to my respected faculty members and mentors for their constant guidance, valuable suggestions, and encouragement throughout the development of this project. Their expert advice and continuous support helped me overcome challenges and improve my understanding of concepts such as linked lists, pointers, and dynamic memory allocation.

I am also thankful to my friends and classmates for their cooperation, helpful discussions, and moral support during the preparation of this project. Their ideas and suggestions contributed to improving the quality of my work.

Lastly, I would like to express my heartfelt thanks to my family for their unconditional support, encouragement, and belief in me throughout my academic journey. Their motivation has always inspired me to perform better and complete my work successfully.

This project has provided me with valuable practical experience and has strengthened my knowledge of C programming and data structures. I am truly grateful to everyone who contributed to this learning experience.

Name: Muskan
UID: 25BCD10182
Course: BCA Data Science

Introduction to Data Structures

Data Structure is a fundamental concept in computer science that refers to the method of organizing, storing, and managing data in a computer system in an efficient and systematic manner. It enables programmers to handle large amounts of data effectively and perform various operations such as insertion, deletion, searching, and updating with ease. In simple terms, a data structure defines how data is arranged in memory and how it can be accessed and modified.

In modern computing, huge volumes of data are generated every day in fields such as education, business, healthcare, and technology. Managing this data efficiently is essential for the smooth functioning of applications. Data structures play a crucial role in achieving this by providing optimized ways to store and process data. The performance of any program largely depends on the type of data structure used, as it directly affects the speed and memory usage of the system.

Data structures are mainly classified into two categories: **linear and non-linear data structures**. Linear data structures are those in which elements are arranged sequentially, and each element is connected to its previous and next element. Examples include arrays, linked lists, stacks, and queues. These are simple to implement and are widely used in basic programming applications. On the other hand, non-linear data structures organize data in a hierarchical or network form, where elements are not arranged sequentially. Examples include trees and graphs, which are used in more complex applications like databases, networking, and artificial intelligence.

Each data structure has its own advantages and limitations. For instance, arrays allow quick access to elements using indexing but have a fixed size, whereas linked lists provide dynamic memory allocation and flexibility but require additional memory for storing pointers. Therefore, choosing the appropriate data structure is very important for solving a particular problem efficiently.

Data structures are widely used in real-life applications such as database management systems, operating systems, web applications, and mobile apps. They help in organizing data in a way that improves efficiency, reduces complexity, and enhances the overall performance of the system.

In this project, data structures are used to develop a **Telephone Contact List Management System**. Specifically, the linked list data structure is implemented to store contact details dynamically. This allows the system to add, delete, search, and update records efficiently without any fixed size limitation.

Thus, data structures form the backbone of programming and software development. A strong understanding of data structures helps in writing efficient programs and solving real-world problems effectively.

Introduction to Arrays and Structures

Arrays and structures are two of the most important and widely used concepts in C programming for organizing and managing data efficiently. They help programmers store, access, and manipulate data in a structured manner, which is essential for developing effective and reliable software applications. Both arrays and structures play a significant role in implementing data structures and solving real-world problems.

An **Array** is a collection of elements of the same data type stored in contiguous memory locations. It allows multiple values to be stored under a single variable name, making the program more organized and easier to manage. Each element in an array is identified by its index, which starts from zero. For example, if we want to store the marks of multiple students, we can use an array instead of declaring separate variables for each student.

Arrays are very useful in situations where a fixed number of elements need to be stored and processed. They provide fast and direct access to elements using index values, which makes operations like retrieval and updating very efficient. Arrays are commonly used in applications such as storing lists of numbers, processing data sets, and implementing algorithms.

There are different types of arrays in C, such as:

- **One-dimensional arrays** – used for storing a list of elements
- **Two-dimensional arrays** – used for storing data in tabular form like matrices
- **Multi-dimensional arrays** – used for complex data representation

Despite their advantages, arrays have certain limitations. The size of an array must be defined at the time of declaration and cannot be changed later. This can lead to memory wastage if the allocated size is larger than required or insufficient if the size is smaller than needed. Also, arrays can only store elements of the same data type, which limits their flexibility.

A **Structure** is a user-defined data type in C that allows grouping of variables of different data types under a single name. It is used to represent real-world entities that consist of multiple attributes. For example, a student record may include name, roll number, marks, and contact details, which can all be stored together using a structure.

Structures are very powerful because they allow combining different types of data into one unit. This makes programs more organized and improves readability. Structures are widely used in applications such as database management, record systems, and file handling.

A structure in C is defined using the struct keyword. For example:

```
struct Student {
   int rollNo;
   char name[50];
   float marks;
};
```

In this example, all student-related data is grouped together in one structure, making it easier to manage.

Structures can also be used with pointers and dynamic memory allocation to create advanced data structures such as linked lists. In a linked list, each node is a structure that contains data and a pointer to the next node. This combination allows dynamic storage of data, where memory is allocated as needed.

In the context of this project, structures are used to represent each contact, which includes fields like name and phone number. These structures are then linked together using pointers to form a linked list. This approach provides flexibility, as new contacts can be added or removed without any limitation on size.

Arrays and structures together form the foundation for building efficient data structures. While arrays are simple and provide fast access, structures offer flexibility and the ability to represent complex data. When combined with linked lists, they enable dynamic and efficient data management.

Overall, understanding arrays and structures is essential for every programmer, as they are the building blocks of more advanced concepts in data structures and software development. Their proper use leads to better program design, improved performance, and effective handling of real-world data.

Why Linked List is Used?

A **Linked List** is one of the most important data structures used in programming when there is a need to manage data dynamically. In this project, **"Telephone Contact List Using Linked List in C"**, the linked list plays a crucial role in storing and managing contact information efficiently. The use of a linked list is preferred over other data structures, especially arrays, because of its flexibility, dynamic nature, and efficient data handling capabilities.

One of the main reasons for using a linked list is its **dynamic size**. In real-life applications like a contact management system, the number of contacts is not fixed. New contacts can be added at any time, and existing ones can be deleted whenever required. Arrays, on the other hand, require a fixed size to be defined at the beginning, which makes them less flexible. If the array size is too small, it cannot store additional data, and if it is too large, it leads to wastage of memory. Linked lists overcome this limitation by allocating memory dynamically during runtime, which allows the list to grow or shrink as needed.

Another important advantage of linked lists is **efficient insertion and deletion of elements**. In arrays, inserting or deleting an element requires shifting other elements, which can be time-consuming, especially for large datasets. In contrast, linked lists allow insertion and deletion operations to be performed easily by simply updating the pointers. This makes linked lists more efficient when frequent modifications are required, as in the case of managing contact records.

Linked lists also provide **better memory utilization**. Since memory is allocated only when required, there is no wastage of unused space. Each node in a linked list is created using dynamic memory allocation, which ensures that memory is used efficiently. This is particularly useful in applications where the amount of data is unpredictable.

Another reason for using linked lists is their **flexibility in data organization**. Linked lists do not require contiguous memory locations, unlike arrays. This means that even if memory is scattered, linked lists can still function properly by connecting nodes using pointers. This feature makes linked lists more adaptable in situations where memory availability is fragmented.

In this project, each contact is represented as a node containing details like name and phone number, along with a pointer to the next node. This structure allows easy traversal of the list, where each contact can be accessed one by one. Operations such as searching, updating, and displaying contacts are performed by traversing the linked list.

Linked lists also improve the **efficiency of search and update operations** in certain scenarios. Although searching in a linked list is sequential, it is still practical for moderate-sized datasets like

contact lists. Additionally, once a node is found, updating its data is straightforward and does not affect other nodes.

Another key reason for using linked lists in this project is to **demonstrate the practical implementation of pointers and dynamic memory allocation in C programming**. Linked lists provide a clear understanding of how memory is managed and how nodes are connected using pointers. This helps students strengthen their programming concepts and develop better problem-solving skills.

However, linked lists also have some limitations. They require extra memory to store pointers, and accessing elements is slower compared to arrays because there is no direct indexing. Despite these disadvantages, linked lists are still preferred in this project because the benefits outweigh the limitations, especially when dealing with dynamic data.

In conclusion, linked lists are used in this project because they provide flexibility, efficient memory usage, and easy data manipulation. They are ideal for applications where the size of data is not fixed and frequent insertion and deletion operations are required. The use of linked lists makes the Telephone Contact List system more efficient, scalable, and practical. It also helps in understanding important programming concepts, making it a suitable choice for this project.

Types of Linked List

A **Linked List** is a dynamic data structure that consists of nodes connected using pointers. Depending on how the nodes are linked and how many pointers are used, linked lists are classified into different types. Each type has its own structure, advantages, and applications.

The main types of linked lists are:

1. Singly Linked List

A **Singly Linked List** is the simplest type of linked list. In this structure, each node contains two parts:

- **Data** (stores the value)
- **Next Pointer** (stores the address of the next node)

The first node is called the **head**, and the last node points to **NULL**, which indicates the end of the list.

Structure:

[Data | Next] → [Data | Next] → [Data | Next] → NULL

Features:

- Traversal is possible in only one direction (forward)
- Simple and easy to implement
- Requires less memory compared to other linked lists

Advantages:

- Easy insertion and deletion
- Efficient memory usage
- Simple structure

Disadvantages:

- Cannot traverse backward
- Searching takes more time (sequential access)

Use in Project:

👉 This project uses a **singly linked list** because it is simple and sufficient for managing contact records.

2. Doubly Linked List

A **Doubly Linked List** is an advanced type where each node contains three parts:

- **Previous Pointer (prev)** → points to the previous node
- **Data**
- **Next Pointer (next)** → points to the next node

Structure:

NULL ← [Prev | Data | Next] ⇄ [Prev | Data | Next] ⇄ [Prev | Data | Next] → NULL

Features:

- Traversal is possible in both directions (forward and backward)
- Each node is connected to both previous and next nodes

Advantages:

- Easier backward traversal
- More flexible navigation
- Efficient for certain applications

Disadvantages:

- Requires more memory (extra pointer)
- More complex to implement

Applications:

- Navigation systems
- Undo/Redo operations
- Browser history

3. Circular Linked List

In a **Circular Linked List**, the last node does not point to NULL. Instead, it points back to the **first node (head)**, forming a loop.

Structure:

```
[Data | Next] → [Data | Next] → [Data | Next]
    ↑_____________________________________|
```

Features:

- No NULL pointer

- Forms a circular loop
- Traversal can start from any node

Advantages:

- Efficient when repeated traversal is required
- No need to start from head every time

Disadvantages:

- Difficult to detect end of list
- More complex logic

Applications:

- Round-robin scheduling
- Multiplayer games
- Continuous systems

4. Doubly Circular Linked List

A **Doubly Circular Linked List** is a combination of doubly and circular linked lists. Each node has:

- **Previous Pointer**
- **Data**
- **Next Pointer**

The last node points to the first node, and the first node points back to the last node.

Structure:

⟷ [Prev | Data | Next] ⇄ [Prev | Data | Next] ⇄ [Prev | Data | Next] ⟷

Features:

- Traversal in both directions
- Circular connection

Advantages:

- Highly flexible
- Efficient traversal in any direction

Disadvantages:

- Complex implementation
- More memory usage

Different types of linked lists are used based on the requirements of the application. The **Singly Linked List** is the most basic and widely used type due to its simplicity and efficiency. In this project, the singly linked list is chosen because it provides all the required operations such as insertion, deletion, searching, and updating while keeping the implementation simple. Understanding the different types of linked lists helps in selecting the appropriate data structure for solving real-world problems effectively.

Basic Operations on Linked List

The following operations are performed in this project:

1. Insertion (Adding a Node)

A new node is created and added to the list. It can be inserted:

- At the beginning
- At the end
- At a specific position

In this project, insertion is done at the beginning for simplicity.

2. Traversal (Displaying Nodes)

Traversal means visiting each node one by one to display or process data.

```
while(temp != NULL) {
    // print data
    temp = temp->next;
}
```

3. Searching

Searching is used to find a specific element in the list by comparing values.

```
void searchContact() {
    char name[50];
    printf("Enter Name to Search: ");
    scanf("%s", name);

    struct Contact* temp = head;

    while(temp != NULL) {
        if(strcmp(temp->name, name) == 0) {
            printf("\nContact Found!\n");
            printf("Name: %s\n", temp->name);
            printf("Phone: %s\n", temp->phone);
            return;
        }
        temp = temp->next;
    }

    printf("\nContact Not Found\n");
}
```

4. Deletion

Deletion removes a node from the list. It requires updating pointers so that the list remains connected.

```
void deleteContact() {
    char name[50];
    printf("Enter Contact Name to Delete: ");
    scanf("%s", name);

    struct Contact *temp = head, *prev = NULL;
```

```
    // Case 1: First node is to be deleted
    if(temp != NULL && strcmp(temp->name, name) == 0) {
        head = temp->next;
        free(temp);
        printf("\nContact Deleted Successfully!\n");
        return;
    }

    // Case 2: Search for the node to delete
    while(temp != NULL && strcmp(temp->name, name) != 0) {
        prev = temp;
        temp = temp->next;
    }

    // If not found
    if(temp == NULL) {
        printf("\nContact Not Found\n");
        return;
    }

    // Delete node
    prev->next = temp->next;
    free(temp);

    printf("\nContact Deleted Successfully!\n");
}
```

5. Updating

Updating modifies the data of a specific node without changing the structure.

```
void updateContact() {
    char name[50];
    printf("Enter Contact Name to Update: ");
    scanf("%s", name);

    struct Contact* temp = head;

    while(temp != NULL) {
        if(strcmp(temp->name, name) == 0) {
            printf("Enter New Phone Number: ");
            scanf("%s", temp->phone);

            printf("\nContact Updated Successfully!\n");
            return;
        }
        temp = temp->next;
    }

    printf("\nContact Not Found\n");
}
```

Advantages of Linked List

- Dynamic size (no fixed limit)
- Efficient insertion and deletion
- Better memory utilization
- No need for contiguous memory

Disadvantages of Linked List

- Extra memory required for pointers
- Sequential access (no direct indexing)
- More complex than arrays

Application of Linked List in This Project

In this project, a linked list is used to manage student/contact records dynamically. Each node represents a student/contact with details like name and phone number.

- **Add Contact → Insert node**
- **Display → Traverse list**
- **Search → Find node**
- **Update → Modify node data**
- **Delete → Remove node**

This makes the system flexible and efficient, especially when the number of records changes frequently.

Real-Life Applications of Linked List

- Contact management systems
- Music playlist
- Browser history
- Memory allocation
- Undo/Redo operations

Conclusion:

Linked lists are powerful data structures that provide flexibility in managing data dynamically. In this project, they play a key role in implementing the Student/Contact Management System efficiently. Understanding linked lists helps in building strong programming skills and solving real-world problems effectively.

Title of the Project

The title of this project is **"Telephone Contact List Using Linked List in C"**. This project is developed to design and implement a simple, efficient, and user-friendly system for managing telephone contacts using the C programming language. The main objective of this project is to store, organize, and manage contact information such as names and phone numbers in a structured and systematic manner.

In today's modern world, people have a large number of contacts that need to be stored and accessed quickly. Managing these contacts manually using diaries or notebooks is not only time-consuming but also prone to errors such as data loss, duplication, and difficulty in searching. With the increasing need for efficient data handling, there is a requirement for a computerized system that can manage contact details easily and accurately. This project provides a solution to this problem by developing a digital contact management system.

The system is implemented using the **C programming language**, which is known for its simplicity and efficiency. It makes use of important programming concepts such as **structures, pointers, and dynamic memory allocation**. The most important data structure used in this project is the **linked list**, which allows dynamic storage of data. Unlike arrays, linked lists do not require a fixed size and can grow or shrink as needed, making them suitable for applications where the number of records is not known in advance.

In this project, each contact is stored as a node in a linked list, where each node contains the contact's name, phone number, and a pointer to the next node. This structure allows efficient operations such as insertion, deletion, searching, and updating of contact records. The system is designed as a menu-driven program, which makes it easy for users to interact with the application and perform various operations.

The project includes essential functionalities such as adding new contacts, displaying all stored contacts, searching for a specific contact, updating contact details, and deleting unwanted contacts. These features make the system practical and useful for everyday use. The use of linked lists ensures that these operations are performed efficiently without the need for shifting data, as required in arrays.

Another important aspect of this project is that it provides practical exposure to students in applying theoretical concepts of data structures and programming. It helps in improving logical thinking, problem-solving skills, and understanding of how dynamic data structures work in real-life applications. This project also highlights the importance of choosing appropriate data structures for efficient data management.

Overall, the **Telephone Contact List Using Linked List in C** project serves as a simple yet powerful application that demonstrates how programming and data structures can be used to solve real-world problems effectively. It not only fulfills academic requirements but also provides a strong foundation for developing more advanced applications such as mobile-based contact systems, database-driven applications, and web-based management systems in the future.

Objectives of the Project

The main objective of the **"Telephone Contact List Using Linked List in C"** project is to develop an efficient and user-friendly system for managing contact information using the concepts of data structures and C programming. This project aims to provide a practical solution for storing, organizing, and manipulating contact details in a systematic way.

One of the primary objectives of this project is to implement the **linked list data structure** for dynamic storage of contacts. Unlike arrays, linked lists allow flexible memory allocation, which makes it possible to add or remove contacts without any limitation on size. This helps in efficient data management and better utilization of memory.

Another important objective is to perform various operations on contact records, such as adding new contacts, searching for existing contacts, updating contact details, deleting unwanted contacts, and displaying all contacts. These operations help in understanding how data structures work in real-life applications and how they can be used to solve practical problems.

The project also aims to improve the programming skills of students by providing hands-on experience in using **C programming concepts** such as structures, pointers, and dynamic memory allocation. It helps in developing logical thinking and problem-solving abilities, which are essential for software development.

In addition, this project focuses on reducing manual effort and improving the efficiency of managing contact information. By replacing traditional methods like maintaining contact lists in notebooks, the system provides a faster, more accurate, and reliable way to handle data.

Another objective is to design a **menu-driven interface** that allows users to interact easily with the system. This ensures that even users with basic computer knowledge can operate the program without difficulty.

Furthermore, the project aims to demonstrate the real-life application of data structures, especially linked lists, in solving everyday problems. It serves as a foundation for developing more advanced systems such as mobile contact applications , database systems, and web-based platforms.

Overall, the objectives of this project are not only to build a functional application but also to enhance understanding of programming concepts, improve technical skills, and provide practical knowledge that can be applied in future software development projects.

Problem Specification / Need of the Project

In the modern digital era, managing personal and professional contact information has become a daily necessity. People maintain a large number of contacts for communication, networking, and various activities. Traditionally, contact details were stored in notebooks, diaries, or simple mobile lists. However, these methods are not efficient when the volume of data increases. Manual record-keeping systems are slow, difficult to maintain, and highly prone to errors, making them unsuitable for handling large amounts of information.

One of the major problems with manual contact management is the **lack of proper organization and accessibility**. When contacts are stored in physical form, searching for a specific contact becomes time-consuming and inconvenient. Users have to go through multiple pages or entries to find the required information. This process not only wastes time but also reduces productivity, especially when quick access to contact details is needed.

Another significant issue is the **difficulty in updating and modifying records**. In manual systems, any change in contact details such as phone numbers requires rewriting or editing existing records, which increases the chances of errors. Similarly, deleting unwanted or outdated contacts is not a straightforward process and may lead to cluttered and unorganized data. Over time, this makes the system inefficient and hard to manage.

There is also a high risk of **data loss and damage** in traditional methods. Physical records can be lost, misplaced, or damaged due to various reasons such as wear and tear, accidents, or environmental factors. Once the data is lost, it is often impossible to recover it. This highlights the need for a more reliable and secure system for managing contact information.

Even when basic computerized systems are used, such as storing data in arrays, there are certain limitations. Arrays require a fixed size to be defined in advance, which makes them less flexible. If the number of contacts exceeds the predefined size, new data cannot be added. On the other hand, if the allocated size is too large, it leads to wastage of memory. Additionally, operations like insertion and deletion in arrays require shifting of elements, which increases complexity and processing time.

To overcome these challenges, there is a need for a more efficient, flexible, and dynamic system for managing contact data. The **Telephone Contact List Using Linked List in C** project is designed to fulfill this need by providing a structured and computerized solution. This system allows users to perform essential operations such as adding new contacts, searching for specific contacts, updating existing records, deleting unwanted entries, and displaying all contacts in an organized manner.

The use of a **linked list data structure** plays a key role in solving the limitations of traditional systems. Linked lists provide dynamic memory allocation, which means memory is allocated only when required. This eliminates the problem of fixed size and allows the system to grow or shrink according to the number of contacts. It also simplifies insertion and deletion operations, as there is no need to shift elements like in arrays. Instead, only pointers are updated, making the process faster and more efficient.

Another important aspect of this project is its ability to provide a **user-friendly interface** through a menu-driven program. This makes it easy for users to interact with the system and perform operations without requiring advanced technical knowledge. The system ensures that data is handled in a structured and organized manner, improving overall efficiency and usability.

From an academic perspective, this project addresses the need to **bridge the gap between theoretical knowledge and practical implementation**. It helps students understand how data

structures such as linked lists, along with concepts like pointers and dynamic memory allocation, can be applied to solve real-world problems. This enhances programming skills, logical thinking, and problem-solving abilities.

Furthermore, the project serves as a **foundation for future development**. It can be extended to include additional features such as storing email addresses, integrating with databases, adding security features, and developing graphical or web-based interfaces. These enhancements can transform the basic system into a complete contact management solution used in real-life applications.

In conclusion, the need for this project arises from the limitations of traditional and basic digital methods of managing contact information. The **Telephone Contact List Using Linked List in C** provides an efficient, reliable, and flexible solution that improves data organization, reduces manual effort, and enhances performance. It not only fulfills academic requirements but also demonstrates the practical use of data structures in solving everyday problems.

Features of the System

The **Telephone Contact List Using Linked List in C** system provides a number of important features that make it efficient, flexible, and user-friendly for managing contact information. These features are designed to perform all essential operations required for handling contacts in a structured and organized manner.

One of the key features of the system is the **ability to add new contacts**. Users can easily enter contact details such as name and phone number, which are then stored in the linked list. This feature allows dynamic insertion of data without any fixed size limitation, ensuring that the system can handle any number of contacts efficiently.

Another important feature is the **display of all contacts**. The system allows users to view all stored contact records in a clear and organized format. This helps users quickly access and review their contact list without confusion. The traversal operation of the linked list is used to display each contact one by one.

The system also provides a **search functionality**, which enables users to find a specific contact by entering the name. This feature is very useful when the contact list becomes large, as it saves time and effort compared to manually searching through records. The system checks each node in the linked list until the required contact is found.

The **update feature** allows users to modify existing contact details. For example, if a phone number changes, the user can update the record without deleting and re-entering the data. This ensures that the contact information remains accurate and up-to-date.

Another essential feature is the **deletion of contacts**. Users can remove unwanted or outdated contacts from the list. This helps in maintaining a clean and organized database by eliminating unnecessary data. The deletion process is efficient because it only requires updating the pointers of the linked list.

The system is designed with a **menu-driven interface**, which makes it easy for users to interact with the program. The menu displays different options such as add, display, search, update, delete, and exit. Users can simply select an option to perform the desired operation. This makes the system simple and user-friendly, even for beginners.

A significant feature of this project is **dynamic memory allocation**. The system uses functions like malloc() to allocate memory at runtime, which ensures that memory is used only when needed. This avoids memory wastage and allows the system to grow dynamically as more contacts are added.

Another advantage of the system is its **efficient data management using linked lists**. Unlike arrays, linked lists do not require contiguous memory and do not have a fixed size. This makes

operations like insertion and deletion faster and more efficient, especially when dealing with large datasets.

The system also ensures **better organization and accuracy of data**. Since all operations are performed through a structured program, the chances of errors are reduced compared to manual methods. The data is stored in an organized way, making it easy to manage and maintain.

Additionally, the system provides **basic error handling**, such as displaying messages when a contact is not found or when the list is empty. This improves the reliability and usability of the program.

Overall, these features make the **Telephone Contact List System** a simple yet powerful tool for managing contact information. It not only simplifies data handling but also demonstrates the practical application of linked lists and programming concepts in solving real-life problems.

Algorithm & Flowchart

An **algorithm** is a step-by-step logical procedure used to solve a problem, while a **flowchart** is a graphical representation of that algorithm using symbols and arrows. Both are essential in program development as they help in understanding the logic, structure, and flow of the program before actual coding begins.

In the **Telephone Contact List Using Linked List in C** project, the algorithm defines how different operations such as adding, displaying, searching, updating, and deleting contacts are performed. The flowchart provides a visual representation of these steps, making it easier to understand the working of the system.

Algorithm

Step 1: Start the Program

The execution of the program begins. The system initializes and prepares to take input from the user.

Step 2: Initialize Linked List

- Set the **head pointer to NULL**
- This indicates that the contact list is initially empty

Step 3: Display Menu Options

The system displays a menu with the following choices:

1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit

Step 4: Take User Input

- The user selects an option by entering a number
- The program reads the input and processes the selected operation

Step 5: Perform Operations

Case 1: Add Contact

- Create a new node using malloc()
- Input name and phone number
- Set the new node's next pointer to the current head
- Update head to point to the new node
- Display success message

Case 2: Display Contacts

- Check if head is NULL
  - If yes → display "No Contacts Available"
  - If no → traverse the list
- Print each contact's name and phone number

Case 3: Search Contact

- Input the name to search
- Traverse the linked list
- Compare each node's name using strcmp()
- If found → display contact details
- If not found → display "Contact Not Found"

Case 4: Update Contact

- Input the name of the contact
- Traverse the list
- If matching contact is found:
  - Enter new phone number
  - Update the value
- If not found → display message

Case 5: Delete Contact

- Input the name to delete
- If the first node matches:
  - Update head pointer
  - Free memory
- Else:
  - Traverse list with prev and temp pointers
  - Adjust links to remove node
  - Free memory
- If not found → display message

Case 6: Exit

- Terminate the program using exit()

Step 6: Repeat Process

- After each operation, return to the menu
- Continue until user selects Exit

Step 7: Stop the Program

The program ends successfully.

Flowchart :

A **flowchart** visually represents the flow of control in the program using standard symbols. It helps in understanding how the program executes step-by-step.

Standard Symbols Used:

- **Oval** → Start/Stop
- **Rectangle** → Process (e.g., Add, Delete)
- **Diamond** → Decision (Choice)
- **Parallelogram** → Input/Output
- **Arrow** → Flow of control

Flowchart Steps:

1. Start
2. Initialize head = NULL
3. Display menu
4. Input user choice
5. Decision box checks choice:
    - If 1 → Add Contact
    - If 2 → Display Contacts
    - If 3 → Search Contact
    - If 4 → Update Contact
    - If 5 → Delete Contact
    - If 6 → Exit
6. Perform selected operation
7. Return to menu
8. Stop when Exit is selected

Flowchart (Text Representation)

You can draw this neatly in your notebook:

```
      Start
        ↓
  Initialize Head = NULL
        ↓
    Display Menu
        ↓
    Enter Choice
        ↓
    ┌──────────────┐
    │  Choice?     │
    └──────────────┘
 ↓  ↓  ↓  ↓  ↓  ↓
Add Display Search Update Delete Exit
 ↓    ↓    ↓    ↓    ↓    ↓
Perform Operation
        ↓
    Return to Menu
        ↓
      Stop
```

Importance of Algorithm and Flowchart:

Importance of Algorithm

- Gives clear **step-by-step solution**
- Helps in **understanding logic**
- Makes coding **easy and organized**
- Helps in **error detection**
- Improves **problem-solving skills**

Importance of Flowchart

- Provides **visual representation** of logic
- Easy to **understand program flow**
- Helps in **finding errors quickly**
- Makes **planning and designing easier**
- Useful for **presentation and explanation**
-

Conclusion:
The algorithm and flowchart provide a clear and systematic approach to designing the **Telephone Contact List System**. They ensure that all operations are logically structured and easy to understand. By following the algorithm and flowchart, the program becomes efficient, organized, and easy to implement.

Implementation – C Source Code

The implementation of the **Telephone Contact List Using Linked List in C** is done using structures and pointers. Each contact is stored as a node in a linked list, and dynamic memory allocation is used to manage memory efficiently. The program is menu-driven, allowing the user to perform various operations such as adding, displaying, searching, updating, and deleting contacts.
Below is the complete C source code:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

// Structure Definition
struct Contact {
   char name[50];
   char phone[15];
   struct Contact* next;
};

// Head Pointer
struct Contact* head = NULL;

// Add Contact
void addContact() {
   struct Contact* newNode = (struct Contact*)malloc(sizeof(struct Contact));

   printf("Enter Name: ");
   scanf("%s", newNode->name);
```

```
    printf("Enter Phone Number: ");
    scanf("%s", newNode->phone);

    newNode->next = head;
    head = newNode;

    printf("\nContact Added Successfully!\n");
}

// Display Contacts
void displayContacts() {
    struct Contact* temp = head;

    if(temp == NULL) {
        printf("\nNo Contacts Available\n");
        return;
    }

    while(temp != NULL) {
        printf("\nName: %s", temp->name);
        printf("\nPhone: %s\n", temp->phone);
        temp = temp->next;
    }
}

// Search Contact
void searchContact() {
    char name[50];
    printf("Enter Name to Search: ");
    scanf("%s", name);

    struct Contact* temp = head;

    while(temp != NULL) {
        if(strcmp(temp->name, name) == 0) {
            printf("\nContact Found!\n");
            printf("Name: %s\nPhone: %s\n", temp->name, temp->phone);
            return;
        }
        temp = temp->next;
    }

    printf("\nContact Not Found\n");
}

// Update Contact
void updateContact() {
    char name[50];
```

```
    printf("Enter Contact Name to Update: ");
    scanf("%s", name);

    struct Contact* temp = head;

    while(temp != NULL) {
        if(strcmp(temp->name, name) == 0) {
            printf("Enter New Phone Number: ");
            scanf("%s", temp->phone);

            printf("\nContact Updated Successfully!\n");
            return;
        }
        temp = temp->next;
    }

    printf("\nContact Not Found\n");
}

// Delete Contact
void deleteContact() {
    char name[50];
    printf("Enter Contact Name to Delete: ");
    scanf("%s", name);

    struct Contact *temp = head, *prev = NULL;

    if(temp != NULL && strcmp(temp->name, name) == 0) {
        head = temp->next;
        free(temp);
        printf("\nContact Deleted Successfully!\n");
        return;
    }

    while(temp != NULL && strcmp(temp->name, name) != 0) {
        prev = temp;
        temp = temp->next;
    }

    if(temp == NULL) {
        printf("\nContact Not Found\n");
        return;
    }

    prev->next = temp->next;
    free(temp);

    printf("\nContact Deleted Successfully!\n");
}
```

```
// Main Function
int main() {
    int choice;

    while(1) {
        printf("\n===== Telephone Contact List =====\n");
        printf("1. Add Contact\n");
        printf("2. Display Contacts\n");
        printf("3. Search Contact\n");
        printf("4. Update Contact\n");
        printf("5. Delete Contact\n");
        printf("6. Exit\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1: addContact(); break;
            case 2: displayContacts(); break;
            case 3: searchContact(); break;
            case 4: updateContact(); break;
            case 5: deleteContact(); break;
            case 6: exit(0);
            default: printf("\nInvalid Choice\n");
        }
    }
}
```

Output/Screenshots:

```
===== Telephone Contact List =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter Choice: 1
Enter Name: Pratham
Enter Phone Number: 7876261496

Contact Added Successfully!

===== Telephone Contact List =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter Choice: 2

Name: Pratham
Phone: 7876261496
```

```
===== Telephone Contact List =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter Choice: 3
Enter Name to Search: Pratham

Contact Found!
Name: Pratham
Phone: 7876261496

===== Telephone Contact List =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
```

```
6. Exit
Enter Choice: 4
Enter Contact Name to Update: Pratham
Enter New Phone Number: 8894777915

Contact Updated Successfully!

===== Telephone Contact List =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter Choice: 5
Enter Contact Name to Delete: Pratham

Contact Deleted Successfully!
```

```
Contact Updated Successfully!

===== Telephone Contact List =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter Choice: 5
Enter Contact Name to Delete: Pratham

Contact Deleted Successfully!

===== Telephone Contact List =====
1. Add Contact
2. Display Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter Choice: 6


...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation

- **Structure** is used to store contact details
- **Linked List** is used for dynamic storage
- **Pointers** connect nodes
- **Functions** perform operations like add, search, delete, update
- **Menu-driven program** makes it user-friendly

Conclusion:

The **Telephone Contact List Using Linked List in C** project successfully demonstrates how data structures can be used to solve real-life problems efficiently. The system provides important features such as adding, displaying, searching, updating, and deleting contact records in an organized way.

The use of a **linked list** makes the system flexible and dynamic, as it allows contacts to be added or removed without any fixed size limitation. This improves memory usage and makes operations like insertion and deletion easier compared to arrays.

This project also helps in understanding key concepts of **C programming**, such as structures, pointers, and dynamic memory allocation. It improves logical thinking and problem-solving skills by providing practical experience.

Overall, the project is simple, efficient, and user-friendly. It fulfills its purpose of managing contact information effectively and can be further enhanced into more advanced applications in the future.

Applications

The **Telephone Contact List Using Linked List in C** project has a wide range of applications in both daily life and various fields of technology. It demonstrates how data structures can be used to manage and organize information efficiently. Although the project is simple, its concept is used in many real-world systems.

One of the primary applications of this system is in **personal contact management**. Individuals can use such systems to store and manage their personal contacts digitally instead of using traditional methods like notebooks or diaries. It allows easy addition, searching, updating, and deletion of contact details, making daily communication more convenient and organized.

Another important application is in **mobile contact systems**. The contact list feature in mobile phones works on similar principles, where contacts are stored, updated, and retrieved efficiently. Although advanced systems use databases, the basic concept of managing records is similar to this project.

This project is also useful in **office and organizational management**. Small businesses or offices can use such systems to maintain employee or client contact details. It helps in keeping records organized and easily accessible, which improves communication and productivity.

Linked list-based systems are widely used in **music playlists**. In music applications, songs are often stored in a linked list-like structure, allowing users to move forward and backward through the playlist. This demonstrates how dynamic data structures are used in entertainment applications.

Another application is in **browser history management**. Web browsers maintain a history of visited pages, which allows users to navigate back and forth. This functionality is similar to linked list operations, especially in doubly linked lists.

The concept used in this project is also applied in **memory management systems**. Operating systems use linked lists to manage free and allocated memory blocks efficiently. This ensures proper utilization of memory and improves system performance.

In addition, linked lists are used in **data processing and scheduling systems**, such as task scheduling and queue management. For example, in round-robin scheduling, circular linked lists are used to manage tasks efficiently.

From an academic perspective, this project is very useful for **learning and understanding data structures**. It helps students gain practical knowledge of how linked lists work and how they can be

applied to solve real-world problems. It also builds a strong foundation for developing more advanced applications.

Furthermore, this project can be extended and applied in **database management systems**, **web applications**, and **mobile applications** by integrating it with modern technologies. This makes it a stepping stone for building complex and scalable systems.

In conclusion, the applications of this project are not limited to contact management only. The concepts used in this system are widely applied in various fields such as software development, system design, and data management. It highlights the importance of data structures in creating efficient and practical solutions for real-life problems.

Another important application of this system is in **educational institutions**, where student or staff contact details need to be managed efficiently. Schools, colleges, and universities can use such systems to store and organize contact information of students, teachers, and administrative staff. This helps in quick communication and easy access to records when needed.

The concept of this project is also useful in **customer relationship management (CRM) systems**. Businesses use CRM software to maintain customer contact details, track interactions, and improve communication. Although advanced systems use databases and web technologies, the basic idea of storing, updating, and managing records is similar to this linked list-based contact management system.

Future Scope

The **Telephone Contact List Using Linked List in C** project is a basic implementation of a contact management system, but it has a wide scope for future enhancements and improvements. The current system is simple and console-based, which makes it suitable for learning purposes. However, it can be further developed into a more advanced and practical application.

One of the major improvements can be the addition of **file handling or database integration**. Currently, the data is stored temporarily in memory and gets deleted when the program is closed. By using file handling or databases like MySQL, the contact information can be stored permanently and retrieved whenever required.

Another important enhancement is the development of a **Graphical User Interface (GUI)**. The current system uses a command-line interface, which may not be very user-friendly for all users. By creating a GUI using tools like Java, Python, or Visual C++, the system can become more interactive, attractive, and easier to use.

The project can also be extended by adding **additional fields** such as email address, home address, date of birth, and other personal details. This will make the system more complete and useful for real-life applications.

Another possible improvement is the implementation of **sorting and advanced searching techniques**. Contacts can be sorted alphabetically or based on phone numbers, and advanced search options can be added to improve efficiency.

The system can also be upgraded to include **security features**, such as password protection or user authentication. This will ensure that only authorized users can access and modify the contact data.

In addition, the project can be converted into a **web-based or mobile application**. This will allow users to access their contacts anytime and from anywhere, making the system more practical and widely usable.

Another future scope is the integration of **backup and recovery features**. This will help in preventing data loss and ensure that important contact information is always safe.

From a technical perspective, the project can be enhanced by using more advanced data structures such as **doubly linked lists** or integrating it with other concepts like stacks and queues for additional functionality.

Overall, the future scope of this project is very wide. With further development and improvements, it can be transformed into a complete and efficient contact management system used in real-world applications.

Another possible enhancement of this project is the integration of **cloud storage and synchronization features**. By connecting the system to cloud platforms, users can store their contact data online and access it from multiple devices such as computers, tablets, and smartphones. This will improve data accessibility, provide real-time updates, and ensure that important contact information is always available and secure, even if the local system is damaged or lost.

Conclusion

The **Telephone Contact List Using Linked List in C** project successfully demonstrates the practical application of data structures in solving real-life problems. The main objective of developing an efficient and user-friendly contact management system has been achieved through the use of linked lists and C programming concepts.

This project highlights the advantages of using a **linked list** over traditional methods such as arrays. The dynamic nature of linked lists allows the system to handle data efficiently without any fixed size limitation. Operations such as insertion, deletion, searching, and updating are performed easily by using pointers, which makes the system flexible and efficient.

The system provides all the essential features required for managing contact records, including adding new contacts, displaying existing contacts, searching for specific entries, updating information, and deleting unwanted records. The menu-driven interface makes the program simple and easy to use, even for beginners.

From a learning perspective, this project has helped in understanding important programming concepts such as **structures, pointers, and dynamic memory allocation**. It has also improved logical thinking and problem-solving skills by providing practical experience in implementing linked lists.

Moreover, the project shows how manual methods of storing contact information can be replaced with an efficient computerized system. It reduces manual effort, minimizes errors, and ensures better organization of data. The system is reliable, easy to maintain, and can be extended further with additional features.

Although the current implementation is basic and console-based, it has great potential for future enhancements such as database integration, graphical user interface, and web or mobile application development. This makes it a strong foundation for building more advanced and real-world systems.

In conclusion, the project is a successful implementation of data structures that not only fulfills academic requirements but also provides practical knowledge and skills that are useful for future software development.

In addition, this project emphasizes the importance of selecting the right data structure for a specific problem. By using a linked list, the system becomes more adaptable to changing data requirements, which is essential in real-life applications where the amount of data is not fixed. This improves the overall performance and efficiency of the system.

The project also provides a clear understanding of how different operations work together to form a complete system. Each function, such as insertion, deletion, searching, and updating, plays a vital role in maintaining the integrity and usability of the contact list. This integration of multiple operations helps in building a well-structured and functional application.

Furthermore, this project serves as a strong foundation for future learning and development. The concepts learned through this implementation can be applied to more complex systems such as database management, web development, and mobile applications. It prepares students for real-world challenges by giving them practical exposure to programming and system design.

References

1. Thareja, Reema. **"Data Structures Using C"**, Oxford University Press.
2. Kanetkar, Yashavant. **"Let Us C"**, BPB Publications.
3. Balagurusamy, E. **"Programming in ANSI C"**, McGraw Hill Education.
4. Kernighan, Brian W., and Ritchie, Dennis M. **"The C Programming Language"**, Prentice Hall.
5. TutorialsPoint. **"Data Structures Using C"**, Available at: https://www.tutorialspoint.com
6. GeeksforGeeks. **"Linked List Data Structure"**, Available at: https://www.geeksforgeeks.org
7. W3Schools. **"C Programming and Data Structures"**, Available at: https://www.w3schools.com
8. Lecture notes and study material provided by **Chandigarh University**, Department of Computer Science & Engineering.
9. Online documentation and examples related to **C programming and linked lists**.

GitHub Repository

GitHub is a widely used platform for version control and code sharing, which allows developers to store, manage, and collaborate on projects efficiently. It is based on Git, a distributed version control system, and provides features such as code hosting, tracking changes, and maintaining different versions of a project.

For this project, the source code of the **Telephone Contact List Using Linked List in C** has been uploaded to GitHub. This makes the project easily accessible, allows future improvements, and helps in sharing the work with others. It also ensures proper documentation and backup of the project files.

The GitHub repository link for this project is provided below:

☞ **GitHub Link:** _____

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.